Capítulo 1: Instalación y Configuración

1.1 Instalación de Git en Windows

Descargar Git

Visita la página oficial de Git: https://git-scm.com

Descarga la última versión para Windows haciendo clic en "Download for Windows"

Proceso de Instalación

- Ejecuta el instalador descargado
- Acepta la licencia y sigue el asistente
- Mantén las opciones por defecto a menos que tengas requisitos específicos
- En "Adjusting your PATH environment", selecciona "Git from the command line and also from 3rd-party software"
- En la opción de línea de comandos, elige "Use Git and optional Unix tools from the Windows Command Prompt"

1.2 Configuración Inicial

Configurar nombre de usuario y email

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu.email@ejemplo.com"
```

Esta configuración es esencial para identificar tus commits en el repositorio.

Configurar el editor por defecto

```
git config --global core.editor "code --wait"
```

Este comando configura Visual Studio Code como editor por defecto.

Ver configuración actual

```
git config --list
```

2.1 Conceptos Fundamentales

¿Qué es el Control de Versiones?

Un sistema de control de versiones (VCS) permite:

- Registrar cambios en archivos a lo largo del tiempo
- Recuperar versiones específicas más adelante
- Colaborar con otros desarrolladores
- Comparar cambios a lo largo del tiempo
- Ver quién modificó por última vez algo que podría estar causando problemas

¿Qué es un Commit?

Un commit es una instantánea (snapshot) del estado de tu proyecto en un momento específico. Es como una fotografía que captura:

- Los cambios realizados en los archivos
- Quién realizó los cambios
- Cuándo se realizaron los cambios
- Por qué se realizaron (a través del mensaje de commit)

Los commits forman la línea de tiempo de tu proyecto, permitiéndote volver a cualquier punto anterior si es necesario.

Estructura de Git

Las Tres Áreas de Git

- **Working Directory (Directorio de Trabajo)**: Donde modificas tus archivos
- **Staging Area (Área de Preparación)**: Donde preparas los cambios que quieres commitear
- **Repository (Repositorio)**: Donde se almacenan los commits

2.2 Primeros Pasos

Crear un Nuevo Repositorio

```
mkdir mi_proyecto
cd mi_proyecto
git init
```

Esto crea un nuevo directorio con un repositorio Git inicializado.

Buenas Prácticas para Mensajes de Commit

```
# Estructura recomendada
tipo(alcance): descripción corta

descripción detallada si es necesaria

# Ejemplos
feat: añade nueva funcionalidad de login
fix: corrige error en la validación de formularios
docs: actualiza documentación de API
style: formato de código sin cambios funcionales
```

💡 Consejos Importantes

- Mantén los commits atómicos (una sola funcionalidad/fix por commit)
- Usa mensajes descriptivos pero concisos
- Sigue un estándar en los mensajes de commit
- Revisa siempre qué archivos están staged antes de hacer commit

2.5 Verificación del Estado

Comandos Básicos de Verificación

```
# Ver el estado actual del repositorio
git status

# Ver el historial de commits
git log

# Ver el historial en formato compacto
git log --oneline

# Ver cambios en archivos
git diff

# Ver cambios en archivos staged
git diff –staged
```

```
# Commit vacío
git commit --allow-empty -m "Commit inicial"
```

Revisar Cambios

```
# Ver cambios no staged
git diff

# Ver cambios staged
git diff --staged

# Ver cambios en un archivo específico
git diff archivo.txt

# Ver cambios entre commits
git diff commit1..commit2

# Ver historial de cambios
git log
git log --oneline
git log --graph --oneline --all
```

3.3 Gestión de Cambios

Deshacer Cambios

```
# Descartar cambios en el directorio de trabajo
git restore archivo.txt

# Deshacer cambios staged (quitar del staging)
git restore --staged archivo.txt

# Revertir un commit (crear nuevo commit que deshace cambios)
git revert commit_hash

# Resetear al último commit (¡Cuidado!)
git reset --hard HEAD

# Resetear a un commit específico
git reset --hard commit_hash
```

```
# Eliminar tag
git tag -d v1.0
```

3.6 Guardado Temporal (Stash)

Uso del Stash

```
# Guardar cambios en el stash
git stash save "Descripción del stash"

# Listar stashes
git stash list

# Aplicar último stash
git stash apply

# Aplicar stash específico
git stash apply stash@{2}

# Eliminar último stash
git stash drop

# Aplicar y eliminar último stash
git stash pop
```

💡 Mejores Prácticas

- Haz commits frecuentes y pequeños
- Usa mensajes de commit descriptivos y consistentes
- Revisa siempre los cambios antes de hacer commit
- Usa stash para cambios temporales
- Crea tags para versiones importantes

Ejercicio Práctico

1. Crea un nuevo repositorio
2. Añade algunos archivos
3. Realiza varios commits
4. Modifica archivos y usa git diff

```
# Eliminar rama (forzado)
git branch -D nombre-rama

# Renombrar rama
git branch -m nombre-antiguo nombre-nuevo
```

4.3 Fusiones (Merge)

Tipos de Fusiones

- **Fast-forward:** Cuando no hay commits divergentes
- **Recursive:** Cuando hay commits divergentes
- **Octopus:** Merge de múltiples ramas

Comandos de Fusión

```
# Fusionar rama en la rama actual
git merge nombre-rama

# Fusión sin fast-forward
git merge --no-ff nombre-rama

# Abortar una fusión con conflictos
git merge --abort

# Ver ramas fusionadas
git branch --merged

# Ver ramas no fusionadas
git branch --no-merged
```

4.4 Rebase

¿Qué es Rebase?

Rebase es una alternativa a merge que permite mantener un historial más limpio y lineal.

```
# Rebase básico
git rebase rama-destino
```

5. Añadir los archivos resueltos
6. Completar el merge o rebase

Ejemplo de Conflicto

```
<<<<<<< HEAD
código de tu rama actual
=======
código de la rama que intentas fusionar
>>>>>>> nombre-rama
```

Mejores Prácticas

- Mantén las ramas pequeñas y enfocadas
- Actualiza frecuentemente tu rama con los cambios de main/master
- Usa nombres descriptivos para las ramas
- Elimina las ramas después de fusionarlas
- Revisa los cambios antes de hacer merge

Ejercicio Práctico

1. Crea una nueva rama feature
2. Realiza algunos cambios y commits
3. Cambia a main y realiza otros cambios
4. Intenta fusionar la rama feature
5. Resuelve los conflictos si aparecen
6. Prueba a hacer un rebase
7. Compara las diferencias entre merge y rebase

Capítulo 5: Trabajo Colaborativo y Flujos de Trabajo

5.1 Git Flow

Estructura de Git Flow

Ramas Principales

- **main/master:** Código en producción

```
# Crear rama corta para feature
git checkout -b feature/quick-fix

# Merge frecuente a trunk
git checkout main
git merge feature/quick-fix
git push origin main
```

5.3 GitHub Flow

Proceso de GitHub Flow

1. Crear rama desde main
2. Realizar cambios y commits
3. Abrir Pull Request
4. Discusión y revisión
5. Deployment (opcional)
6. Merge a main

```
# Crear rama para feature
git checkout -b feature/nueva-funcionalidad

# Pushear rama y crear PR
git push -u origin feature/nueva-funcionalidad

# Después de la revisión, merge a main
git checkout main
git merge feature/nueva-funcionalidad
git push origin main
```

5.4 Estrategias de Colaboración

Pull Requests / Merge Requests

- Descripción clara del cambio
- Referencias a issues relacionados
- Screenshots o GIFs (si aplica)
- Checklist de cambios
- Asignación de revisores

```
- Configura OAuth2
- Añade manejo de tokens
- Actualiza tests"
```

Sincronización

```
# Mantener repositorio actualizado
git fetch --all
git pull --rebase origin main

# Limpiar ramas obsoletas
git remote prune origin
git branch --merged | grep -v "\*" | xargs -n 1 git branch -d
```

Recomendaciones para Trabajo en Equipo

- Establecer convenciones de commit claras
- Documentar el flujo de trabajo elegido
- Mantener las ramas actualizadas
- Realizar code reviews exhaustivos
- Usar herramientas de calidad de código
- Mantener una comunicación clara en PRs e issues

Ejercicio Práctico

1. Configura un repositorio con Git Flow
2. Crea una feature siguiendo el flujo completo
3. Realiza un release
4. Simula un hotfix
5. Practica la revisión de código con un compañero
6. Implementa un flujo de CI básico

Capítulo 6: Visual Studio Code y Git

6.1 Configuración Inicial

Extensiones Recomendadas

- **GitLens:** Información detallada sobre el historial de Git

Comandos de la Paleta

```
// Comandos comunes (Ctrl/Cmd + Shift + P)
Git: Add Remote
Git: Checkout to...
Git: Create Branch...
Git: Pull
Git: Push
Git: Sync
```

6.4 Gestión de Ramas

Operaciones con Ramas

- Crear nueva rama
- Cambiar entre ramas
- Fusionar ramas
- Eliminar ramas

Accesos Directos

```
# Desde la barra de estado
Click en nombre de rama actual → Lista de ramas

# Desde la paleta de comandos
Git: Checkout to...
Git: Create Branch...
Git: Delete Branch...
```

6.5 Resolución de Conflictos

Herramientas de Merge

- Editor de conflictos visual
- Opciones de Accept Current/Incoming/Both
- Vista de diferencias en tres vías
- Marcadores de conflicto interactivos

💡 Consejos Avanzados

- Usa la vista de Timeline para ver el historial de un archivo
- Configura alias personalizados para comandos frecuentes
- Utiliza los marcadores de Git en el scroll bar
- Aprovecha la integración con GitHub para PRs
- Usa snippets para mensajes de commit estandarizados

🔄 Ejercicio Práctico

1. Instala y configura las extensiones recomendadas
2. Practica el flujo de trabajo básico usando la interfaz de VS Code
3. Crea y gestiona ramas desde la interfaz
4. Resuelve un conflicto usando el editor de merge
5. Explora las características de GitLens
6. Personaliza la configuración según tus necesidades

Capítulo 7: GitHub y Repositorios Remotos

7.1 Configuración Inicial de GitHub

Creación de Cuenta y Configuración

1. Registrarse en GitHub
2. Configurar perfil
3. Configurar SSH keys
4. Configurar autenticación de dos factores

Configuración de SSH

```
# Generar nueva clave SSH
ssh-keygen -t ed25519 -C "tu_email@ejemplo.com"

# Iniciar el agente SSH
eval "$(ssh-agent -s)"

# Añadir la clave al agente
ssh-add ~/.ssh/id_ed25519
```

7.3 Pull Requests

Proceso de Pull Request

1. Crear rama para feature
2. Realizar cambios y commits
3. Pushear rama a GitHub
4. Crear Pull Request
5. Revisión de código
6. Aplicar cambios sugeridos
7. Merge del PR

Buenas Prácticas para PRs

```
# Plantilla de Pull Request

## Descripción
[Descripción breve de los cambios]

## Cambios Realizados
- [Cambio 1]
- [Cambio 2]
- [Cambio 3]

## Testing
- [ ] Test unitarios
- [ ] Test de integración
- [ ] Test manuales

## Screenshots
[Si aplica]

## Issues Relacionados
Closes #123
```

7.4 Issues y Proyectos

Gestión de Issues

- Crear issues descriptivos

7.6 Características Avanzadas

GitHub Pages

```
# Configurar GitHub Pages
# 1. Ir a Settings > Pages
# 2. Seleccionar branch source
# 3. Elegir carpeta (root o /docs)
# 4. Save
```

GitHub Packages

- Publicación de paquetes
- Gestión de dependencias
- Versionado semántico

Gists

- Compartir snippets de código
- Versionar snippets
- Embedder en otras páginas

7.7 Seguridad y Permisos

Configuraciones de Seguridad

- Gestión de accesos
- Revisión de dependencias
- Análisis de código
- Configuración de branch protection

Branch Protection Rules

```
# Reglas comunes
- Require pull request reviews
- Require status checks to pass
- Require signed commits
- Include administrators
- Allow force pushes
- Allow deletions
```

8.2 Anatomía de un Conflicto

Marcadores de Conflicto

```
<<<<<<< HEAD
const greeting = "Hola Mundo";
=======
const greeting = "Hello World";
>>>>>>> feature/english-version
```

Significado de los Marcadores

- `<<<<<<< HEAD`: Inicio del conflicto - versión actual
- `=======`: Separador entre versiones
- `>>>>>>> rama`: Final del conflicto - versión de la rama

8.3 Herramientas de Resolución

Herramientas en Línea de Comandos

```
# Ver estado de archivos en conflicto
git status

# Ver diferencias en detalle
git diff

# Marcar como resuelto
git add <archivo>

# Continuar merge después de resolver
git merge --continue
```

Herramientas Visuales

- VS Code Git
- GitKraken
- Meld
- Beyond Compare
- P4Merge

Estrategias de Branching

```
# Actualizar rama antes de empezar
git checkout feature-branch
git pull origin main

# Rebase periódico
git checkout feature-branch
git rebase main

# Merge de mantenimiento
git checkout feature-branch
git merge main
```

8.6 Escenarios Complejos

Conflictos en Rebase

```
# Durante un rebase
git rebase main

# Si hay conflictos
# 1. Resolver conflictos
# 2. git add <archivos>
# 3. git rebase --continue

# Para abortar
git rebase --abort
```

Conflictos en Cherry-Pick

```
# Al aplicar commit específico
git cherry-pick <commit-hash>

# Resolver conflictos
git add <archivos>
git cherry-pick --continue

# Para abortar
git cherry-pick --abort
```

- **Trees:** Directorios y metadata
- **Commits:** Snapshots del proyecto
- **Tags:** Referencias nombradas

Comandos Internos

```
# Ver objetos Git
git cat-file -p <hash>

# Examinar estructura interna
git ls-tree HEAD

# Ver objetos sueltos
git fsck

# Mantenimiento y optimización
git gc
git prune
```

9.2 Git Hooks Avanzados

Hooks Personalizados

```
#!/bin/bash
# .git/hooks/pre-commit
# Validación de estilo de código
eslint .

# Ejecutar tests
npm test

# Verificar mensajes de commit
#!/bin/bash
# .git/hooks/commit-msg
commit_regex='^(feat|fix|docs|style|refactor|test|chore):.+'
error_msg="El mensaje de commit debe seguir Conventional Commits"

if ! grep -iqE "$commit_regex" "$1"; then
    echo "$error_msg" >&2
    exit 1
fi
```

```
# Clon parcial
git clone --filter=blob:none repositorio

# Limpiar historial
git filter-branch --tree-filter 'rm -rf carpeta-grande' HEAD

# Comprimir repositorio
git gc --aggressive --prune=now
```

Git LFS (Large File Storage)

```
# Configurar LFS
git lfs install

# Trackear archivos grandes
git lfs track "*.psd"
git lfs track "*.zip"

# Ver archivos trackeados
git lfs ls-files
```

9.5 Seguridad Avanzada

Firma de Commits y Tags

```
# Configurar GPG
git config --global user.signingkey <key-id>

# Firmar commits
git commit -S -m "commit firmado"

# Firmar tags
git tag -s v1.0.0 -m "Release v1.0.0"

# Verificar firmas
git verify-commit <commit-hash>
git verify-tag v1.0.0
```

```
        npm ci
        npm run build
    - name: Create Release
      uses: actions/create-release@v1
      env:
        GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
      with:
        tag_name: ${{ github.ref }}
        release_name: Release ${{ github.ref }}
```

9.7 Mejores Prácticas Profesionales

Estructura de Commits

- Commits atómicos y coherentes
- Mensajes descriptivos y estandarizados
- Referencias a issues/tickets
- Separación de cambios refactor/feature

Flujo de Trabajo en Equipo

- Code review sistemático
- Documentación actualizada
- Tests automatizados
- Integración continua
- Despliegue controlado

Mantenimiento a Largo Plazo

- Limpieza periódica de ramas
- Actualización de dependencias
- Monitoreo de tamaño del repositorio
- Backups regulares
- Documentación de procesos

🎓 Conclusiones del Manual

- Git es una herramienta fundamental en el desarrollo moderno
- La práctica constante es clave para el dominio
- Las mejores prácticas evolucionan con el tiempo

```
git checkout -b feature/header

# En index.html añadir:
<header>
    <nav>
        <ul>
            <li><a href="#">Inicio</a></li>
            <li><a href="#">Servicios</a></li>
            <li><a href="#">Contacto</a></li>
        </ul>
    </nav>
</header>

git commit -am "feat: añade estructura del header"
```

A.2 Ejercicio: Simulación de Conflictos

Escenario

Simularemos un conflicto común en el desarrollo y aprenderemos a resolverlo.

```
# Developer 1
git checkout -b feature/estilos-header
# Modificar styles.css
header {
    background: blue;
    color: white;
}

# Developer 2
git checkout -b feature/header-responsive
# Modificar styles.css
header {
    background: navy;
    color: lightgray;
}

# Resolución
git checkout main
git merge feature/estilos-header
git merge feature/header-responsive
```

```
# Crear release
git flow release start v1.0.0
# Ajustes finales
git flow release finish v1.0.0
```

A.4 Ejercicio: Automatización con GitHub Actions

Crear Pipeline de CI/CD

```yaml
# .github/workflows/deploy.yml
name: Deploy Blog

on:
  push:
    branches: [ main ]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2

      - name: Setup Node.js
        uses: actions/setup-node@v2
        with:
          node-version: '14'

      - name: Install dependencies
        run: npm install

      - name: Build
        run: npm run build

      - name: Deploy
        run: |
          # Comandos de despliegue
```

a. Implementar Git Flow
b. Configurar GitHub Pages
c. Automatizar despliegue

2. Aplicación de Notas

a. Trabajo en equipo
b. Resolución de conflictos
c. Feature branches

3. API Documentation

a. Versionado semántico
b. Release management
c. Documentación automática

💡 Consejos para los Ejercicios

- Usa un repositorio de práctica para experimentar
- Documenta los comandos que uses
- Simula diferentes escenarios de error
- Practica la recuperación de errores
- Trabaja con otros desarrolladores cuando sea posible

🔧 Herramientas Recomendadas

- GitKraken - Visualización de ramas
- Visual Studio Code + GitLens
- GitHub Desktop para principiantes
- Git Graph extension
- Git Command Explorer